PROJECT PRESENTATION & TECHNICAL VIVA DEFENSE MANUAL

CreditRiskSystem — Viva Guide & Presentation Topics

In-Depth Guide covering Architecture Concepts, SCSS Fundamentals, Live Data Pipelines, DOM Animation Directives, Responsive Web Design, Code Walkthroughs, and Model Answers for Presentation Defense.

`SCSS vs CSS` `Live Market Ingestion` `IntersectionObserver` `Angular 18 Architecture` `Responsive Design (RWD)` `Financial Calculators` `Admin Security` `Code Snippets`

ADMIN PORTAL ROUTE

`/admin` (Frontend) • `/api/admin/login` (Backend)

ADMIN PASSWORD

`Admin@2025!` (Configured in appsettings.json)

PRIMARY FRAMEWORK

Angular 18+ Standalone with TypeScript Signals

STYLING PARADIGM

Custom SCSS (Glassmorphic Dark Mode, Flexbox, Grid)

1 Admin Portal Access & Credentials

🔑 **Administrator Login Details**

- **Frontend Navigation Route:** `/admin` (Access via browser: `http://localhost:4200/admin` or live deployment).
- **Backend API Endpoint:** `POST /api/admin/login` (Handled by `AdminController.cs`).
- **Admin Username:** No username required (Single-credential institutional role).
- **Admin Password:** `Admin@2025!` (Defined in `appsettings.json` under `AdminSettings:Password`).
- **Show / Hide Password Feature:** Now integrated directly on the admin login card with an interactive SVG eye toggle!

When you enter `Admin@2025!` and submit, the server generates an HMAC-SHA256 signed **JSON Web Token (JWT)** valid for 8 hours (480 minutes). The token is saved in browser `sessionStorage` under the key `'admin-token'`. Every subsequent administrative request asynchronously includes `Authorization: Bearer <token>` in its HTTP headers.

2 Understanding SCSS (Sassy Cascading Style Sheets)

What do you mean by SCSS, and why did we use it instead of regular CSS?

SCSS (Sassy CSS) is a powerful CSS preprocessor that extends standard CSS with programmatic superpowers: nesting, variables, mixins, math operations, and modular imports. Browsers cannot understand SCSS directly, so during the build process (`ng build`), the Angular Sass compiler transpiles all `.scss` files into standard, highly optimized, minified CSS (`styles.css`).

Feature	Standard CSS Limitation	How SCSS Solves It in Our Project
Selector Nesting	Must repeat parent class names over and over: `.login-card { }` `.login-card .form-input { }`	Clean visual hierarchy using nesting and the BEM parent selector (`&`): `.login-card { &__icon { } &__title { } }`
Design System Tokens	Native CSS variables exist, but SCSS variables allow compile-time constants, math, and color functions.	We combine both! SCSS compiles global color maps and breakpoint variables with zero runtime overhead.
Hover & State Styling	Requires writing separate rules: `.journey-card--teal:hover { }`	Nested pseudo-classes: `&--teal:hover { border-color: var(--accent-teal); }`
Modularity	Loading multiple CSS files causes multiple HTTP network roundtrips.	Component-scoped SCSS (`styleUrl: './home.scss'`) bundles styles directly with the component JS chunk.

3 How Live Market Data is Fetched

How is the live market information fetched and displayed on the home page?

The live market data pipeline is a multi-tier architecture spanning public macroeconomic APIs, ASP.NET Core server-side caching, and Angular asynchronous HttpClient polling:

1. Server-Side Data Ingestion (`MarketDataServices.cs`)

- **USD / INR Exchange Rate:** Fetched from Frankfurter Open Currency API: `https://api.frankfurter.app/latest?from=USD&to=INR`.
- **Nifty 50 Index:** Fetched live via Yahoo Finance Chart API: `^NSEI?interval=1d&range=1d`.
- **BSE Sensex:** Fetched live via Yahoo Finance Chart API: `^BSESN?interval=1d&range=1d`.
- **Macroeconomic Policy Rates:** RBI Repo Rate (6.50%), Inflation (4.85%), and Gold Price (₹72,000/10g) are cached and can be manually updated in real-time by the Admin.

2. Client-Side Presentation (`HomeComponent`)

- When `HomeComponent` initializes (`ngOnInit()`), it calls `financeService.getMarketSnapshot()`.
- The Angular `HttpClient` makes an asynchronous AJAX GET call to `/api/market/snapshot`.
- The response populates the `marketData` Signal.
- The collapsible floating widget on the homepage displays green/amber live badges and auto-formats numbers using Angular's `DecimalPipe`.

- **In-Memory Caching:** Cached for 24 hours to prevent API rate-limiting.

4 Scroll Reveal: How Elements Appear on Scrolling

When scrolling down the page, different elements appear with smooth animations. How is this implemented?

This is implemented using a custom Angular Directive: `ScrollRevealDirective` (`directives/scroll-reveal.ts`). Instead of using heavy third-party JavaScript libraries or lagging `window.onscroll` event listeners that slow down the browser, we use the browser's native, hardware-accelerated `IntersectionObserver` **API**.

Here is the exact code and mechanism from `scroll-reveal.ts`:

```
// 1. Initial State: Element is hidden and translated downwards
private setInitialState(el: HTMLElement) {
  el.style.transition = 'opacity 0.6s ease, transform 0.6s ease';
  el.style.opacity    = '0';
  if (this.revealDirection === 'fadeUp')    el.style.transform = 'translateY(32px)';
  if (this.revealDirection === 'fadeLeft')  el.style.transform = 'translateX(-32px)';
  if (this.revealDirection === 'fadeRight') el.style.transform = 'translateX(32px)';
}

// 2. IntersectionObserver watches when element enters viewport
this.observer = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      setTimeout(() => {
        element.style.opacity   = '1';
        element.style.transform = 'none'; // Smoothly returns to natural position!
      }, this.revealDelay);
      this.observer.unobserve(element); // Disconnects observer to save memory
    }
  });
}, { threshold: 0.12, rootMargin: '0px 0px -40px 0px' });
```

How we attach it to HTML elements:
We simply add the directive attribute to any card or section: `<div class="journey-card" scrollReveal revealDirection="fadeUp" [revealDelay]="200">`.

5 How the Different Calculators Work

Calculator	Mathematical Principle	Working Mechanism in Code
SIP Calculator (Investment Page)	Future Value of Annuity `FV = P * [((1+i)^n - 1) / i] * (1+i)`	Takes monthly deposit P, converts annual return to monthly compounding rate $i = r/12$, and calculates future wealth, total invested capital, and total wealth gain over n months.
CAGR Calculator (Investment Page)	Compound Annual Growth `(V_final / V_initial)^(1/t) - 1`	Calculates the exact smoothed geometric annual growth rate of any asset given initial investment, final value, and number of years.
NAV Monte Carlo (Investment Page)	Geometric Brownian Motion (GBM) `dS_t = μS_t dt + σS_t dW_t`	Runs 500 stochastic simulation paths using drift (μ), annual volatility (σ), and random Gaussian increments (dW). Calculates median expected NAV, 10th percentile worst-case, and 90th percentile bull-case.
Loan Risk Profiler (Apply Page)	5-Factor Quantitative Engine (Sharpe + CAPM + Kelly + VaR + Merton)	Computes borrower default probability, calculates 30-day Value-at-Risk exposure, and evaluates Distance-to-Default (Merton model). Combines them into a 0–100 Composite Score: **Approve (≥70)**, **Review (40–69)**, **Reject (<40)**.
Loan vs. Fund (Compare Page)	Sharpe-Adjusted Opportunity Cost Analysis	Pits borrowing interest outflow against investing the same capital in an equity fund. Compares net wealth created and Sharpe ratios, recommending whether taking a loan or investing is mathematically superior.

6 Navigation Menu & Responsive Design (Aspect Ratios)

What do we call the menu that we added?

The menu is called a **Sticky Glassmorphic Navigation Bar** on desktop, which dynamically adapts into an **Interactive Mobile Drawer / Hamburger Menu** on smaller screens.

- **Sticky Behavior:** Uses Angular Signals (`scrolled = signal(false)`) listening to window scroll. When scrolled past 20px, it applies the class `navbar--scrolled`, adding a dark frosted glass effect (`backdrop-filter: blur(20px)`).
- **Active Link Highlighting:** Uses Angular's `routerLinkActive="active"` to automatically highlight the current page.
- **Hamburger Toggle:** On screens below 768px, navigation links collapse, and an animated 3-bar hamburger icon appears (`navbar__burger`). Clicking it triggers `toggleMenu()`, sliding out the mobile drawer.

How have we implemented responsive design across different screen sizes and aspect ratios?

We implemented **Responsive Web Design (RWD)** without any heavy third-party frameworks through a multi-tiered CSS architecture:

1. **CSS Media Queries:** Targeted breakpoints at 1200px (desktop), 900px (tablets/laptops), 768px (tablets), and 500px (mobile):

```
.stats__grid {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr)); // 4 columns on desktop
  @media (max-width: 900px) { grid-template-columns: repeat(2, minmax(0, 1fr)); } // 2 cols
  @media (max-width: 500px) { grid-template-columns: 1fr; } // 1 col on mobile
}
```

2. **CSS Grid with Auto-Fit:** Fluid card containers use `repeat(auto-fit, minmax(280px, 1fr))`, allowing cards to automatically reorganize without manual media queries.
3. **CSS Flexbox with Wrap:** Action bars and form controls use `flex-wrap: wrap`, preventing horizontal overflow on narrow screens.
4. **Relative & Viewport Units:** Using `rem`, `vh`, and percentage widths instead of hardcoded pixel sizes.
5. **Vector Scalability (SVG):** All icons use SVG with `viewBox="0 0 24 24"`, rendering razor-sharp on Retina and standard screens.

7 Journey Cards Hover Effects: How and Why

Hovering on different journey cards on the homepage shows different colored borders. How and why?

WHY (Design Rationale & UX): Each journey card represents a completely different financial workflow:

- **Teal Border:** Loan Risk Profiler (`/apply`) — Teal communicates stability, security, and institutional credit trust.
- **Amber Border:** Investment Tools (`/invest`) — Amber/Gold symbolizes wealth growth, compounding returns, and capital accumulation.
- **Purple Border:** Loan vs. Fund Comparison (`/compare`) — Purple represents strategic intelligence and analytical decision-making.

This color-coding creates immediate visual hierarchy and gives the user tactile micro-feedback.

HOW (CSS Implementation in `home.scss`):

```
.journey-card {
  transition: border-color 0.2s ease, transform 0.2s ease;

  &:hover {
    transform: translateY(-2px); // Subtle elevation
  }

  &--teal:hover   { border-color: var(--accent-teal);   } // #14b8a6
  &--amber:hover  { border-color: var(--accent-amber);  } // #f59e0b
  &--purple:hover { border-color: var(--accent-purple); } // #8b5cf6
}
```

In HTML, Angular renders the dynamic class: `<div class="journey-card journey-card--{{ j.color }}">`.

8 Why Use Angular and How Did It Help?

Why choose Angular over React, Vue, or Vanilla JavaScript, and how did it help our project?

Angular 18 was chosen because financial enterprise software demands strict architectural structure, compile-time safety, and high-performance reactive state management:

1. **Single Page Application (SPA) Performance:** Users navigate between Loan Profiling, Investment Calculators, Model Learning, and Admin Dashboard with **zero page refreshes**. Navigation is instantaneous.
2. **Angular 18 Signals:** Provides fine-grained reactive state management (`signal()`, `computed()`) with micro-targeted DOM re-renders, giving 60fps performance without Zone.js overhead.
3. **First-Class TypeScript Integration:** Catching numerical type mismatch errors at compile-time instead of having runtime financial calculation failures.
4. **Built-In Dependency Injection & HttpClient:** Out-of-the-box AJAX architecture with RxJS Observables, eliminating the need to install external libraries like Axios.
5. **Standalone Component Architecture:** Eliminates bloated `NgModule` files, enabling modular, lazy-loaded route bundles (as seen in our build output: `admin.js`, `apply.js`, `invest.js`).

9 Code Snippets & Angular Implementation Examples

9.1 Real JavaScript / TypeScript Calculation Snippet (`apply.ts`)

```
// Dynamic EMI & Risk Calculation in TypeScript:
get debtToIncomeRatio(): number {
  return this.form.annualIncome > 0
    ? Math.min(this.form.existingDebt / this.form.annualIncome, 1)
    : 0;
}

get monthlyEMI(): number {
  const r = (this.form.interestRate / 100) / 12; // Monthly interest rate
  const n = this.form.loanTermYears * 12;        // Total months
  if (r === 0) return this.form.loanAmount / n;
  // Standard Amortization Formula: P * r * (1+r)^n / ((1+r)^n - 1)
  return (this.form.loanAmount * r * Math.pow(1 + r, n)) / (Math.pow(1 + r, n) - 1);
}
```

9.2 Complete Angular Triad Architecture (Component TS + HTML + SCSS)

Here is how a feature is structured from end-to-end in CreditRiskSystem:

TypeScript Component (`admin.ts`)

```
@Component({
  selector: 'app-admin',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './admin.html',
  styleUrl: './admin.scss'
})
export class AdminComponent {
  private http = inject(HttpClient);
  password = '';
  showPassword = signal(false); // Reactive Signal
  isLoggedIn = signal(false);

  login() {
    this.http.post<any>(`${API}/login`, { password: this.password })
      .subscribe({
        next: (res) => {
          sessionStorage.setItem('admin-token', res.token);
          this.isLoggedIn.set(true);
        }
      });
  }
}
```

HTML Template (`admin.html`)

```
<!— Two-Way Binding & Signal-Driven Type Toggle —>
<div class="password-input-wrap">
  <input class="form-input"
         [type]="showPassword() ? 'text' : 'password'"
         placeholder="Enter password"
         [(ngModel)]="password"
         (keyup.enter)="login()">

  <button type="button" class="password-toggle-btn"
          (click)="showPassword.update(v => !v)">
    @if (showPassword()) {
      <svg ...><!— Eye Off Icon —></svg>
    } @else {
      <svg ...><!— Eye On Icon —></svg>
    }
  </button>
</div>
```


Ready for Evaluation • All Rights Reserved